

At a conceptual level, all boards are programmed over an RS-232 serial connection. Current Arduino boards include the USB-to-serial adapter chips such as the FTDI FT232. The Arduino board exposes most of the microcontroller's I/O pins for use by other circuits. The Diecimila (another kind of Arduino board), for example, provides 14 digital I/O pins, six of which can produce PWM signals, and six analogue inputs. The Arduino boards available in the market are the Arduino Uno, Nano, Mega, Leonardo, Due, etc. The main differences between these are the controller used, and the total number of output pins and memory available for programming. We can choose particular boards according to the application and user requirements.

The core Arduino developer team includes Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino, David Mellis and Nicholas Zambetti.

Writing one's own Arduino library

Libraries are files written in either C or C++, which provide the sketch/source code with extra or common functionality. The library file has three different file formats — *.cpp, *.h and one txt file that contains the different keywords used in library creation. The *.h file should contain the header files and *.cpp will hold the programming for that particular library.

In this example, our library EFYLM35 has *EFYLM35.cpp*, *EFYLM35.h*, and *keywords.txt* files. After writing these three files, they are placed in a zip file. To use a prebuilt existing library in our Arduino program, go to the *Sketch* menu on the IDE, choose *Import Library* and pick from the libraries available as EFYLM35. That will insert our library design *#include EFYLM35.h* and inbuilt Arduino IDEs *#include Arduino.h* at the top of the sketch along with the imported header file. These statements make the public functions and constants defined by the library available to our sketch. They also signal the Arduino IDE to link that library's code with our code when it is compiled or uploaded. Because libraries are uploaded on the board with our sketch, they increase the amount of space used by the ATmega8 on the board. If we no longer need a particular library, simply delete its *#include* statements from the top of our code. This will stop the Arduino IDE from linking the library and reduce the amount of space used on the Arduino board.

The basic design steps for a user library on the Arduino IDE are listed below.

Step 1: The basic sketch for the library is written and the code layout is as follows:

```
#include <EFYLM35.h> //The custom made library for LM35
class EFYLM35 {Statements} //declaring EFYLM35 class objects
void setup () {EFYLM35.initial(); //initializing the class definition}
void loop () {EFYLM35.proceed(); //things to at every loop iteration}
```

In the above code, function *initial ()* is for initialising the variables (ports, etc) and the *proceed ()* function is for some operational work at each looping cycle. Now make a list of all the input and output parameters we need. It's time to step up and write our own Arduino library for the above operation. Arduino can also be built up in Java.

Step 2: To build the library, you are required to create two files—one with the extension *EFYLM35.h* and the second with the extension *EFYLM35.cpp*. In the *EFYLM35.h* (header file), the general structure is:

```
#ifndef __EFYLM35__
#define __EFYLM35__ //include types & constants of Wiring core API
class EFYLM35 {public: //variables & functions, accessible outside of the class, and the class constructor
EFYLM35(); //Default constructor of class EFYLM35
void initial (); //define first function
void proceed (); //define second function
private: //private variables & functions
int input process (); //to process and draw conclusion from input
}; //end of class xyz definition
#endif //ending with end if
```

Step 3: Save the above code as *EFYLM35.h*. The next step is to write your own .cpp file, which will contain the *main()* function, which is the driver function in a C++ program. Save this file as *EFYLM35.cpp*. Take both these files under one zip folder, and our own custom-made library.

```
#include<EFYLM35.h> //including the custom made header file
EFYLM35:: EFYLM35 () { //calling the default constructor}
void EFYLM35:: initial () { //the body of initial () function
//setup your sensor here}
void EFYLM35::proceed() { //the body of the proceed()}
int input_data= input_process(); }
int EFYLM35::input_process() { //analogRead function}
int EFYLM35:: EFYLM35 (int inputData) { //some complex stuff for LM35 conversion and output the results }
```



Figure 2: Interfacing LM 35 with Arduino

The program

The types and constraints of the application program interface (API) are defined by *#ifndef __EFYLM35__* and *#define __EFYLM35__*.

To access the I/O functionalities of Arduino,

we have to import the Arduino library by using the header file *#include Arduino.h*. Here, we have defined one class as the name *EFYLM35*. This class declares constructors and functions for reading the temperature value as well as to perform the averaging of a few samples to increase the